第六周作业-2

第六周作业

提高作业

2.1 右臂运动学脚本

> 1. 参考 `arm_kinematics.py`，编写一个支持右臂 FK 和 IK 的 Python 脚本
> 2. 查阅 URDF 确定右肩在 torso 坐标系中的位置，找出与左臂的参数差异
> 3. 用 `tf_echo /torso /right_hand` 验证脚本输出是否正确，提交测试结果

1. *支持右臂 FK 和 IK 的 Python 脚本见最后附件*
2. *根据 URDF 描述，可以知道右肩在 torso 坐标系中的位置为 (-0.13, 0, 0.35)，其他没有差异。*
3. *验算 IK 操作，移动末端点到 (-0.13, 0.1, 0.2)，运行 python 文件，得到如下结果（下图 1）；与 `tf_echo /torso /right_hand` （下图 2）的结果相同*

1.
```
[sunzj@SH3 ~/Robot/ws_gazebo]$ rosrun humanoid_sim arm_kinematics_right.py ik 0.1 0.2
====================================================
目标: s = 0.1000 m, d = 0.2000 m
q1 = 1.3347 rad (76.5 deg)  限位OK
q2 = -1.8872 rad (-108.1 deg)  限位OK
FK 验证: s=0.1000, d=0.2000, 误差=0.000000 m
末端 torso 坐标: x=-0.130, y=0.1000, z=0.1500

>>> 可直接粘贴到终端发送给机器人:
rostopic pub /humanoid/position_controller/command std_msgs/Float64MultiArray "data: [0.0, 1.3347, 0.0, -1.
8872, 0.0, 0.0, 0.0, 0.0]" -1
====================================================
```

2.
```
At time 5912.677
- Translation: [-0.130, 0.100, 0.150]
- Rotation: in Quaternion [-0.272, 0.000, 0.000, 0.962]
            in RPY (radian) [-0.551, 0.000, 0.000]
            in RPY (degree) [-31.594, 0.000, 0.000]
```

2.2 旋转表示相互转换

> 1. 阅读《机器人学导论》第二章关于空间描述与变换的内容
> 2. 编写一个 Python 程序，实现以下四种转换功能：
> - 旋转矩阵 → RPY 欧拉角
> - 旋转矩阵 → 四元数
> - RPY 欧拉角 → 旋转矩阵
> - 四元数 → 旋转矩阵
> 3. 用 `tf_echo` 输出中的 Rotation（同时提供了四元数和 RPY）作为测试数据，验证你的转换程序是否正确

1. *Python 转换程序见最后的附录 2*
2. *测试得到的结果如下：*

> = 测试: RPY -> 旋转矩阵 -> RPY =
> 原始 RPY: (0.5, 0.3, -0.2)

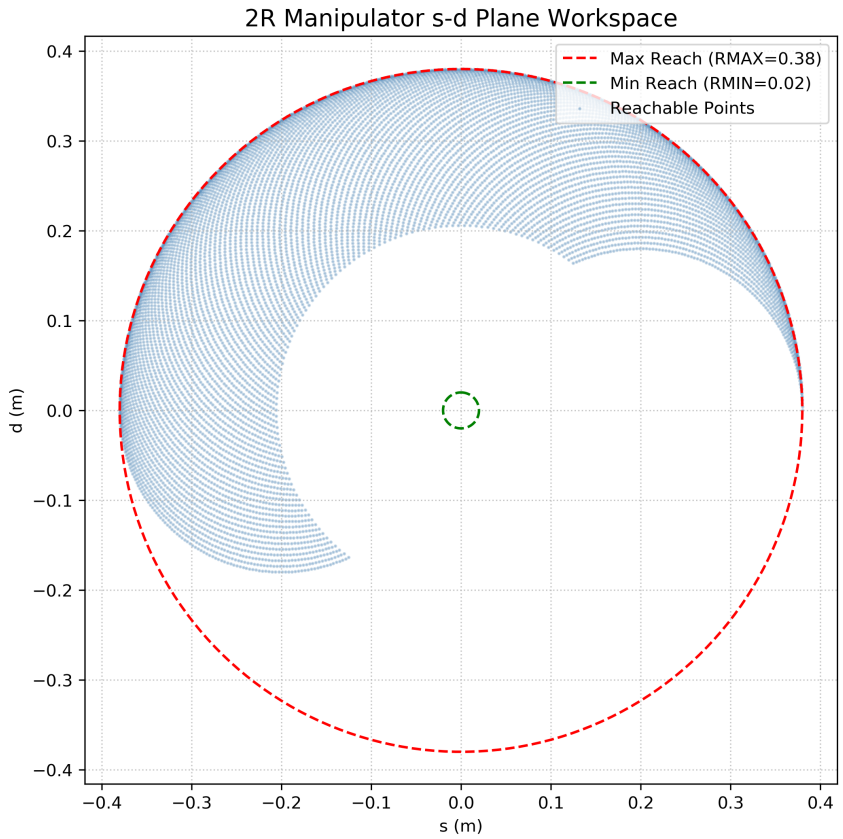
```
转换后的旋转矩阵 R:
[[ 0.9363 0.3132 0.1589]
[-0.1898 0.8319 -0.5214]
[-0.2955 0.458 0.8384]]
恢复的 RPY: [ 0.5 0.3 -0.2]

= 测试: 旋转矩阵 -> 四元数 -> 旋转矩阵 =
转换后的四元数 [w, x, y, z]: [ 0.9496 0.2579 0.1196 -0.1324]
恢复的旋转矩阵 R:
[[ 0.9363 0.3132 0.1589]
[-0.1898 0.8319 -0.5214]
[-0.2955 0.458 0.8384]]
```

2.3 可达工作空间可视化

1. 编写 Python 脚本，遍历关节限位范围内的 q1、q2 组合（各取 100 个等间距值），用 FK 公式计算末端坐标，用 matplotlib 绘制 s-d 平面散点图
2. 在图中标注最大可达距离和最小可达距离的圆弧
3. 分析为什么可达区域不是一个完整的圆环，提交思考结果

1&2. Python 程序见最后附录 3，散点图如下：



3. 是由于 q1 和 q2 的关节限位导致可达区域不是一个完整的圆环。q1 被限制在了 [-1.57, 1.57] 角度范围内，而 q2 倍限制在了 [-2, 0] 范围内，导致无法形成完整的圆。

提高作业提交要求

> 将以上内容整理为 **一份 PDF**，包含：
>
> 1. 右臂脚本完整代码 + 与 tf_echo 的对比验证截图
> 2. 旋转转换脚本完整代码 + 用 tf_echo 数据验证的运行结果截图
> 3. 工作空间可视化脚本完整代码 + 生成的工作空间图片
> 4. 各题思考分析

附录 1. 右臂脚本完整代码

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
第六讲 2R 右臂运动学工具
所有连杆参数与 humanoid_kinematics.urdf 完全一致。

右臂与左臂的参数差异（均来自 URDF）：
  - 肩关节在 torso 系 x 坐标：右臂 -0.13 m（左臂 +0.13 m）
  - rostopic 关节索引：右臂 [0,q1,0,q2,...] vs 左臂 [q1,0,q2,0,...]
  - right_elbow 限位 upper=0（左臂 upper=2.0），解B(q2>0) 恒不可用

用法:
  arm_kinematics_right.py fk <q1> <q2>              正运动学
  arm_kinematics_right.py ik <s> <d>                逆运动学（2R 平面坐标）
  arm_kinematics_right.py ik --torso <y> <z>        逆运动学（torso 坐标系）
  arm_kinematics_right.py reach <s> <d>             可达性检查
  arm_kinematics_right.py multi <s> <d>             多解对比（肘弯曲 vs 肘反
向）

示例:
  arm_kinematics_right.py fk 0.8 -0.9
  arm_kinematics_right.py ik 0.10 0.30
  arm_kinematics_right.py ik --torso -0.15 0.10
  arm_kinematics_right.py reach 0.10 0.50
  arm_kinematics_right.py multi 0.10 0.30
"""
import math
import sys

# ==================== URDF 参数 ====================
L1 = 0.20   # 肩关节 → 肘关节
L2 = 0.18   # 肘关节 → 手端（right_hand）
SHOULDER_IN_TORSO = (-0.13, 0.0, 0.35)
Q1_LIMITS = (-1.57, 1.57)
Q2_LIMITS = (-2.0, 0.0)
RMAX = L1 + L2
RMIN = abs(L1 - L2)

def fk_2r(q1, q2):
```

```
    """2R 平面 FK → (s, d)"""
    s = L1 * math.sin(q1) + L2 * math.sin(q1 + q2)
    d = L1 * math.cos(q1) + L2 * math.cos(q1 + q2)
    return s, d

def to_torso(s, d):
    """(s, d) → torso 坐标系 (x, y, z)
    右臂: x = SHOULDER_IN_TORSO[0] = -0.13, y = s (侧向与左臂相同), z = 0.35
- d
    """
    return SHOULDER_IN_TORSO[0], s, SHOULDER_IN_TORSO[2] - d

def rostopic_cmd(q1, q2):
    return (f'rostopic pub /humanoid/position_controller/command '
            f'std_msgs/Float64MultiArray '
            f'"data: [0.0, {q1:.4f}, 0.0, {q2:.4f}, 0.0, 0.0, 0.0, 0.0]"
-1')

# ==================== fk 子命令 ====================
def cmd_fk(args):
    q1 = float(args[0]) if len(args) > 0 else 0.8
    q2 = float(args[1]) if len(args) > 1 else -0.9

    s, d = fk_2r(q1, q2)
    x, y, z = to_torso(s, d)

    print("=" * 55)
    print(f"输入: q1 = {q1:.4f} rad ({math.degrees(q1):.1f} deg)")
    print(f"      q2 = {q2:.4f} rad ({math.degrees(q2):.1f} deg)")
    print()
    print(f"手端 (torso 系): x={x:.4f}, y={y:.4f}, z={z:.4f}")
    print(f"  ↑ 应与 tf_echo /torso /right_hand 的 Translation 一致")
    print(f"  (右肩在 torso x = {SHOULDER_IN_TORSO[0]:.3f} m, 与左臂符号相
反)")
    print()
    print(f"2R 平面坐标: s={s:.4f} (侧向), d={d:.4f} (下垂)")
    print("=" * 55)
    print()
    print(">>> 对应的关节命令:")
    print(rostopic_cmd(q1, q2))

# ==================== ik 子命令 ====================
def cmd_ik(args):
    if len(args) >= 3 and args[0] == "--torso":
        y_t, z_t = float(args[1]), float(args[2])
        s_target = y_t
        d_target = SHOULDER_IN_TORSO[2] - z_t
        print(f"torso 目标: y={y_t:.3f}, z={z_t:.3f}")
        print(f"转换 → 2R 平面: s={s_target:.3f}, d={d_target:.3f}")
        print()
```

```
    elif len(args) >= 2:
        s_target, d_target = float(args[0]), float(args[1])
    else:
        s_target, d_target = 0.10, 0.30

    r2 = s_target ** 2 + d_target ** 2
    c2 = (r2 - L1 ** 2 - L2 ** 2) / (2 * L1 * L2)

    if abs(c2) > 1.0:
        r = math.sqrt(r2)
        print(f"目标点不可达: |c2| = {abs(c2):.4f} > 1")
        print(f"目标距离 = {r:.4f} m, 可达范围 = [{RMIN:.2f}, {RMAX:.2f}]
m")
        sys.exit(1)

    q2 = -math.acos(c2)
    q1 = math.atan2(s_target, d_target) - math.atan2(
        L2 * math.sin(q2), L1 + L2 * math.cos(q2)
    )

    s_fk, d_fk = fk_2r(q1, q2)
    err = math.sqrt((s_fk - s_target) ** 2 + (d_fk - d_target) ** 2)
    _, y_fk, z_fk = to_torso(s_fk, d_fk)

    q1_ok = Q1_LIMITS[0] <= q1 <= Q1_LIMITS[1]
    q2_ok = Q2_LIMITS[0] <= q2 <= Q2_LIMITS[1]

    print("=" * 55)
    print(f"目标: s = {s_target:.4f} m, d = {d_target:.4f} m")
    print(f"q1 = {q1:.4f} rad ({math.degrees(q1):.1f} deg)  限位{'OK' if
q1_ok else 'FAIL'}")
    print(f"q2 = {q2:.4f} rad ({math.degrees(q2):.1f} deg)  限位{'OK' if
q2_ok else 'FAIL'}")
    print(f"FK 验证: s={s_fk:.4f}, d={d_fk:.4f}, 误差={err:.6f} m")
    print(f"末端 torso 坐标: x={SHOULDER_IN_TORSO[0]:.3f}, y={y_fk:.4f}, z=
{z_fk:.4f}")
    print()
    if q1_ok and q2_ok:
        print(">>> 可直接粘贴到终端发送给机器人:")
        print(rostopic_cmd(q1, q2))
    else:
        print(">>> 超出关节限位，无法执行")
    print("=" * 55)

# ==================== reach 子命令 ====================
def cmd_reach(args):
    s = float(args[0]) if len(args) > 0 else 0.10
    d = float(args[1]) if len(args) > 1 else 0.50

    r = math.sqrt(s ** 2 + d ** 2)
```

```
    c2 = (s ** 2 + d ** 2 - L1 ** 2 - L2 ** 2) / (2 * L1 * L2)

    print(f"目标: s = {s}, d = {d}")
    print(f"目标到肩关节距离: r = {r:.4f} m")
    print(f"可达范围: [{RMIN:.2f}, {RMAX:.2f}] m")
    print(f"c2 = {c2:.4f}")
    print()
    if abs(c2) > 1.0:
        print(f"|c2| = {abs(c2):.4f} > 1 → 目标点不可达")
        if r > RMAX:
            print(f"原因: 目标距离 {r:.3f} > 最大可达距离 {RMAX:.2f}")
        elif r < RMIN:
            print(f"原因: 目标距离 {r:.3f} < 最小可达距离 {RMIN:.2f}")
    else:
        print("目标点可达")

# ==================== multi 子命令 ====================
def cmd_multi(args):
    s_target = float(args[0]) if len(args) > 0 else 0.10
    d_target = float(args[1]) if len(args) > 1 else 0.30

    r2 = s_target ** 2 + d_target ** 2
    c2 = (r2 - L1 ** 2 - L2 ** 2) / (2 * L1 * L2)

    if abs(c2) > 1.0:
        print("目标点不可达")
        sys.exit(1)

    print(f"目标: s = {s_target}, d = {d_target}")
    print()

    for name, sign in [("解 A (q2 < 0, 肘弯曲)", -1),
                       ("解 B (q2 > 0, 肘反向)", +1)]:
        q2 = sign * math.acos(c2)
        q1 = math.atan2(s_target, d_target) - math.atan2(
            L2 * math.sin(q2), L1 + L2 * math.cos(q2)
        )
        s_fk, d_fk = fk_2r(q1, q2)

        q1_ok = Q1_LIMITS[0] <= q1 <= Q1_LIMITS[1]
        q2_ok = Q2_LIMITS[0] <= q2 <= Q2_LIMITS[1]

        print(f"{name}:")
        print(f"  q1 = {q1:+.4f} rad ({math.degrees(q1):+.1f} deg)  "
              f"限位{'OK' if q1_ok else 'FAIL'}")
        print(f"  q2 = {q2:+.4f} rad ({math.degrees(q2):+.1f} deg)  "
              f"限位{'OK' if q2_ok else 'FAIL'}")
        print(f"  FK check: s={s_fk:.4f}, d={d_fk:.4f}")
        if q1_ok and q2_ok:
            print(f"  >>> 可用:")
```

```
            print(f"  {rostopic_cmd(q1, q2)}")
        else:
            print(f"  >>> 超出 URDF 关节限位，不可用")
        print()

# ==================== 主入口 ====================
COMMANDS = {
    "fk": cmd_fk,
    "ik": cmd_ik,
    "reach": cmd_reach,
    "multi": cmd_multi,
}

if __name__ == "__main__":
    if len(sys.argv) < 2 or sys.argv[1] not in COMMANDS:
        print(__doc__)
        sys.exit(0)
    COMMANDS[sys.argv[1]](sys.argv[2:])
```

附录 2. RPY 欧拉角与旋转矩阵与四元数的互相转换

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
旋转表示方法的相互转换：
- 旋转矩阵 ↔ RPY 欧拉角 (Roll-Pitch-Yaw, 对应 X-Y-Z 轴旋转)
- 旋转矩阵 ↔ 四元数 (Quaternion, w, x, y, z)
"""
import numpy as np
import math

def rpy_to_rot(r, p, y):
    """
    RPY 欧拉角 -> 旋转矩阵
    RPY (Roll, Pitch, Yaw) 分别对应绕 X, Y, Z 轴的旋转角。
    ROS 中默认的 tf 转换采用的是固定轴 X-Y-Z (相当于连体轴 Z-Y-X)。
    R = R_z(yaw) * R_y(pitch) * R_x(roll)
    """
    # 绕 X 轴
    Rx = np.array([
        [1, 0, 0],
        [0, math.cos(r), -math.sin(r)],
        [0, math.sin(r), math.cos(r)]
    ])
    # 绕 Y 轴
    Ry = np.array([
        [math.cos(p), 0, math.sin(p)],
        [0, 1, 0],
```

```
        [-math.sin(p), 0, math.cos(p)]
    ])
    # 绕 Z 轴
    Rz = np.array([
        [math.cos(y), -math.sin(y), 0],
        [math.sin(y), math.cos(y), 0],
        [0, 0, 1]
    ])

    R = Rz.dot(Ry).dot(Rx)
    return R

def rot_to_rpy(R):
    """
    旋转矩阵 -> RPY 欧拉角
    """
    # 检查万向锁情况 (Pitch = +/- 90度)
    sy = math.sqrt(R[0, 0]**2 + R[1, 0]**2)
    singular = sy < 1e-6

    if not singular:
        r = math.atan2(R[2, 1], R[2, 2])
        p = math.atan2(-R[2, 0], sy)
        y = math.atan2(R[1, 0], R[0, 0])
    else:
        # 万向锁
        r = math.atan2(-R[1, 2], R[1, 1])
        p = math.atan2(-R[2, 0], sy)
        y = 0

    return r, p, y

def quat_to_rot(q):
    """
    四元数 -> 旋转矩阵
    输入 q 格式为: [w, x, y, z] 或 [x, y, z, w]
    这里假设输入格式为 ROS 常用格式: [x, y, z, w] (为了通用，下面以参数 w,x,y,z
解析)
    由于不同库格式不同，我们显式传入 [w, x, y, z] 的顺序列组或对象。
    但这里我们为了防止混淆，设定输入为 [w, x, y, z] (标量在前)
    """
    w, x, y, z = q

    # 归一化四元数以防计算误差
    norm = math.sqrt(w**2 + x**2 + y**2 + z**2)
    w, x, y, z = w/norm, x/norm, y/norm, z/norm

    R = np.array([
        [1 - 2*y**2 - 2*z**2,     2*x*y - 2*w*z,     2*x*z + 2*w*y],
        [    2*x*y + 2*w*z, 1 - 2*x**2 - 2*z**2,     2*y*z - 2*w*x],
```

```
        [    2*x*z - 2*w*y,     2*y*z + 2*w*x, 1 - 2*x**2 - 2*y**2]
    ])
    return R

def rot_to_quat(R):
    """
    旋转矩阵 -> 四元数 [w, x, y, z]
    """
    trace = np.trace(R)

    if trace > 0:
        s = 0.5 / math.sqrt(trace + 1.0)
        w = 0.25 / s
        x = (R[2, 1] - R[1, 2]) * s
        y = (R[0, 2] - R[2, 0]) * s
        z = (R[1, 0] - R[0, 1]) * s
    else:
        if R[0, 0] > R[1, 1] and R[0, 0] > R[2, 2]:
            s = 2.0 * math.sqrt(1.0 + R[0, 0] - R[1, 1] - R[2, 2])
            w = (R[2, 1] - R[1, 2]) / s
            x = 0.25 * s
            y = (R[0, 1] + R[1, 0]) / s
            z = (R[0, 2] + R[2, 0]) / s
        elif R[1, 1] > R[2, 2]:
            s = 2.0 * math.sqrt(1.0 + R[1, 1] - R[0, 0] - R[2, 2])
            w = (R[0, 2] - R[2, 0]) / s
            x = (R[0, 1] + R[1, 0]) / s
            y = 0.25 * s
            z = (R[1, 2] + R[2, 1]) / s
        else:
            s = 2.0 * math.sqrt(1.0 + R[2, 2] - R[0, 0] - R[1, 1])
            w = (R[1, 0] - R[0, 1]) / s
            x = (R[0, 2] + R[2, 0]) / s
            y = (R[1, 2] + R[2, 1]) / s
            z = 0.25 * s

    return np.array([w, x, y, z])

if __name__ == "__main__":
    print("=== 测试: RPY -> 旋转矩阵 -> RPY ===")
    rpy_original = (0.5, 0.3, -0.2)
    print(f"原始 RPY: {rpy_original}")

    R = rpy_to_rot(*rpy_original)
    print(f"转换后的旋转矩阵 R:\n{np.round(R, 4)}")

    rpy_recovered = rot_to_rpy(R)
    print(f"恢复的 RPY: {np.round(rpy_recovered, 4)}\n")

    print("=== 测试: 旋转矩阵 -> 四元数 -> 旋转矩阵 ===")
```

```
    quat = rot_to_quat(R)
    print(f"转换后的四元数 [w, x, y, z]: {np.round(quat, 4)}")

    R_recovered = quat_to_rot(quat)
    print(f"恢复的旋转矩阵 R:\n{np.round(R_recovered, 4)}")
```

附录 3 可达工作空间可视化

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
import numpy as np
import matplotlib.pyplot as plt
import math

def main():
    # 连杆参数
    L1 = 0.20
    L2 = 0.18
    # 关节限位范围（参考左臂或基础参数）
    Q1_MIN, Q1_MAX = -1.57, 1.57
    Q2_MIN, Q2_MAX = -2.0, 0

    # 获取 100 个等间距值
    q1_vals = np.linspace(Q1_MIN, Q1_MAX, 100)
    q2_vals = np.linspace(Q2_MIN, Q2_MAX, 100)

    # 遍历所有 q1, q2 组合计算末端坐标 s, d
    S = []
    D = []
    for q1 in q1_vals:
        for q2 in q2_vals:
            # 根据 2R 正运动学计算
            s = L1 * math.sin(q1) + L2 * math.sin(q1 + q2)
            d = L1 * math.cos(q1) + L2 * math.cos(q1 + q2)
            S.append(s)
            D.append(d)

    # 计算极值圆弧
    rmax = L1 + L2
    rmin = abs(L1 - L2)

    # 绘制图片
    plt.figure(figsize=(8, 8))
    # 散点图
    plt.scatter(S, D, s=1, c='steelblue', alpha=0.3, label='Reachable
Points')

    # 绘制全圆或圆弧（为了完全表示覆盖面，这里画全圆参照线）
    theta = np.linspace(0, 2 * math.pi, 200)
```

```
    plt.plot(rmax * np.sin(theta), rmax * np.cos(theta), 'r--',
linewidth=1.5, label=f'Max Reach (RMAX={rmax:.2f})')
    plt.plot(rmin * np.sin(theta), rmin * np.cos(theta), 'g--',
linewidth=1.5, label=f'Min Reach (RMIN={rmin:.2f})')

    # 图表细节
    plt.title("2R Manipulator s-d Plane Workspace", fontdict={'family':
'sans-serif', 'size': 14})
    plt.xlabel("s (m)")
    plt.ylabel("d (m)")
    plt.axis("equal")
    plt.grid(True, linestyle=':', alpha=0.7)

    # 解决中文显示可能出现的方块问题（如果系统中存在问题可以注释掉，用纯英文替代）
    # plt.rcParams['font.sans-serif'] = ['SimHei', 'DejaVu Sans']
    # plt.rcParams['axes.unicode_minus'] = False

    # 让 matplotlib 自动获取我们在 plot 和 scatter 中设定的 label
    plt.legend(loc="upper right")
    # 保存结果到本地而不阻塞终端
    save_path =
"/home/sunzj/Robot/ws_gazebo/src/humanoid_sim/scripts/workspace_plot.png"
    plt.savefig(save_path, dpi=300, bbox_inches='tight')
    print(f"工作空间绘制完成，图片已保存至: {save_path}")

if __name__ == "__main__":
    main()
```